



Recetas Bolivianas App

Práctica: Integración Continua y Despliegue Continuo

GitHub Actions · Node.js · React · Render · Vercel

Objetivos de la práctica

Al finalizar esta práctica, el estudiante será capaz de:

- Crear un repositorio en GitHub y subir un proyecto full-stack
- Interpretar un archivo de pipeline YAML de GitHub Actions
- Observar en tiempo real la ejecución de tests automáticos con Jest
- Verificar que un build de React se genera correctamente en el pipeline
- Configurar un deploy automático a Render (backend) y Vercel (frontend)
- Demostrar cómo un test fallido bloquea el deploy a producción

💡 Concepto clave: CI/CD significa que cada vez que un desarrollador sube código, el sistema automáticamente verifica que funciona (CI) y lo publica en producción (CD) — sin intervención manual.

Arquitectura del proyecto

Estructura de carpetas

```
recetas-app/
├── .github/workflows/ci-cd.yml  ← Pipeline (el corazón del CI/CD)
├── backend/
│   ├── src/app.js              ← API REST con Express
│   ├── src/recetas.js          ← Datos (array de recetas)
│   └── tests/recetas.test.js    ← Tests automáticos (Jest)
└── frontend/
    └── src/App.jsx              ← Interfaz React (Vite)
```

Rutas de la API

Método	Ruta	Descripción
GET	/api/recetas	Devuelve todas las recetas
GET	/api/recetas/:id	Una receta por ID (404 si no existe)
GET	/api/recetas/categoria/:cat	Filtrar por almuerzo / desayuno

Procedimiento paso a paso

Fase 1: Configurar el repositorio

1

Crear cuenta en GitHub

Ir a github.com → Sign up. Usar el correo institucional. Verificar el correo.

2

Crear repositorio

En GitHub: New repository → Nombre: recetas-app → Public → Create repository

3

Clonar y subir el proyecto

Ejecutar en la terminal de tu máquina Windows (Git Bash o cmd con Git instalado):

```
git clone https://github.com/TU_USUARIO/recetas-app.git
cd recetas-app
# Copia todos los archivos del proyecto aquí
git add .
git commit -m "feat: proyecto inicial recetas app"
git push origin main
```

Fase 2: Observar el pipeline en acción

4

Ver GitHub Actions ejecutarse

En tu repositorio → pestaña Actions → click en el workflow en ejecución

🔗 Observar: verás 3 jobs en paralelo. Los jobs test-backend y build-frontend corren al mismo tiempo. El job deploy solo corre si los dos anteriores pasaron exitosamente.

5

Revisar los logs de tests

Click en el job 'Backend — Tests' → expandir el paso 'Ejecutar tests' → leer la salida de Jest

Deberías ver algo como:

```
PASS tests/recetas.test.js
  API Recetas
    ✓ GET /api/recetas - retorna lista de recetas (45ms)
    ✓ GET /api/recetas/:id - retorna receta existente (12ms)
    ✓ GET /api/recetas/:id - retorna 404 si no existe (8ms)
    ✓ GET /api/recetas/categoria/:cat - filtra por categoría (10ms)

Tests: 4 passed, 4 total
```

Fase 3: Deploy en la nube

6

Crear cuenta en Render (backend)

Ir a render.com → New → Web Service → conectar repositorio GitHub → Root: backend
→ Build: npm install → Start: npm start

7

Crear cuenta en Vercel (frontend)

Ir a vercel.com → New Project → conectar repositorio → Root: frontend → Framework:
Vite → agregar variable VITE_API_URL con la URL de Render

8

Configurar Secrets en GitHub

Settings → Secrets and variables → Actions → New repository secret

Secret	Valor
VITE_API_URL	https://tu-backend.onrender.com
RENDER_DEPLOY_HOOK_URL	URL obtenida desde Render → Settings → Deploy Hook

Fase 4: Demostrar el valor de CI/CD

Experimento A — Cambio exitoso

Agrega una receta nueva al array en backend/src/recetas.js, luego:

```
git add . && git commit -m "feat: agregar receta tucumanas" && git push
```

Observar: el pipeline corre, tests pasan, deploy automático.

Experimento B — Test fallido bloquea el deploy

Edita backend/tests/recetas.test.js, cambia una línea para que falle:

```
expect(res.statusCode).toBe(999); // falla a propósito
git add . && git commit -m "test: forzar fallo" && git push
```

Observar: el job test-backend falla en rojo, el deploy NO se ejecuta.



Conclusión del experimento B: Este es el valor central de CI/CD. El sistema detecta el error automáticamente y protege producción. En un equipo real, el desarrollador recibe una notificación y arregla el bug antes de que llegue a usuarios.

Preguntas de evaluación

Responder en el informe de la práctica:

1. ¿Qué diferencia hay entre Integración Continua (CI) y Despliegue Continuo (CD)?
2. ¿Por qué el job deploy tiene la propiedad needs: [test-backend, build-frontend]?
3. ¿Qué son los Secrets de GitHub y por qué no se escriben directamente en el archivo YAML?
4. ¿Qué pasaría si eliminamos la condición if: github.ref == 'refs/heads/main' del job deploy?
5. Explica con tus palabras qué ventaja tiene usar CI/CD en un proyecto de software real.

Entregable

- URL del repositorio GitHub con el pipeline funcionando
- Captura de pantalla del pipeline en verde (todos los jobs pasados)
- Captura de pantalla del pipeline en rojo (experimento B — test fallido)
- URL de la app desplegada en Vercel
- Respuestas a las 5 preguntas de evaluación (mínimo 3 líneas cada una)